

TDDC17 Artificial Intelligence

Introduction to PDDL

PDDL is a standardized "Planning Domain Definition Language" which is widely used to describe planning domains as well as problem instances. It was initially developed for use in an International Planning Competition (IPC) in 1998, allowing all competing planners to use the same input language, and has been gradually but significantly extended and polished since that time. Currently almost all new planners support some subset of PDDL.

Levels of Expressivity: STRIPS, ADL, and Others

The PDDL language supports many different levels of expressivity. As you will see later, any domain definition must explicitly declare its expressivity requirements in the shape of a `:requirements` clause. This allows a PDDL parser to verify that you only use the expressivity you have explicitly requested, and allows a planner to easily tell you whether you are requesting features it does not support.

Unfortunately, many planners' parsers ignore this specification...

The most basic level of expressivity is called STRIPS, approximately corresponding to what was supported by the **STanford Research Institute Problem Solver** in 1971 (this is the planner that was used for the Shakey robot). To request this level of expressivity, say `(:requirements :strips)` – details below.

A later language called ADL, **Action Description Language**, added support for a number of different extensions to STRIPS: Negative preconditions (you can only move from A to B if you are NOT carrying anything), disjunctive preconditions (you can only travel by train if you have money OR you already have a ticket), disjunctive goals (you want to own a car OR a motorcycle), and a few other extensions. You can request full ADL expressivity using `(:requirements :adl)`.

Many planners do not support all of the features of ADL. The reason for this is that creating plans efficiently is quite difficult, requiring clever internal plan structures and search spaces. Some search spaces work very well given certain limitations on expressivity, but cannot easily be extended to work outside those limitations. For example, in forward state space search (10.2.1 in the book) we always have a complete state in which a precondition should be evaluated and therefore supporting disjunctive preconditions is trivial. On the other hand, if a planner is based on backward state space search (10.2.2), disjunctive preconditions lead to an explosion in the search space, as regression no longer returns a unique preceding state but a *set* of potential preceding states.

It is still quite common that planners support *some* extensions to plain STRIPS, though. PDDL allows us to request such extensions at a fine level of granularity, as in `(:requirements :strips :disjunctive-preconditions)`.

For those of you who want to go straight to the source, here is the **IPC 1998 specification of PDDL**. This version should be taken with a grain of salt, however. The **IPC 2000 specification** is significantly reduced, and closer to what most planning systems actually support. The specification for **IPC 2002** (often called *PDDL 2.1*) adds many new features, including numerical state variables, actions with duration and the possibility to specify a metric to optimize in addition to the logical problem goal. However, there are still many planners not supporting these extensions.

Examples

We provide **several examples** of PDDL domain and problem definitions using only the STRIPS subset of PDDL as well as **several examples** using object types and some ADL features.

We also provide several variants of the **Satellite domain** used in IPC 2002, which illustrate the extended features of PDDL 2.1.

Tools

For those of you who use Emacs (and know how to customize it), there is a special **PDDL mode** you can download. This mode provides syntax highlighting for PDDL. Otherwise, you can use Scheme mode (M-x scheme-mode) or Lisp mode (M-x lisp-mode), or simply plain text mode (no changes required).

A parser is available which you can optionally use to test whether your PDDL domain definitions are syntactically correct. *Note that the parser requires the same `planners.mod` as the planners. See the instructions on running the planners.*

```
parser domainFile [problemFile]
```

The parser can check the domain or the domain and problem file. It cannot check the problem file without the domain. Be aware that the parser can be very pedantic and not always give you understandable error messages. It is advised to check the domain with a planner like FF first, and if you cannot figure out what an error message is, or it runs but you get weird results instead, then the parser can be used as a last resort. Some of the terms the parser uses is covered in a follow-up course on planning (TDDD48).

The PDDL Domain Definition

A PDDL definition consists of two parts: The *domain definition* and the *problem (instance) definition*. Although not required by the PDDL standard, many planners require that the two parts are in separate files.

The domain definition gives each domain a name and specifies the predicates and operators that are available in the domain (though operators are called *actions* in PDDL). It may also specify types (see **Typing**, below), constants, static facts and many other things, some of which are not supported by the majority of planners.

The format of a (simple) domain definition is as follows. Elements in [] are optional. You may want to keep one or two of the **example domain definitions** open while you read this.

```
(define (domain DOMAIN_NAME)
  (:requirements [:strips] [:equality] [:typing] [:adl] ...)
  [(:types T1 T2 T3 T4 ...)]
  (:predicates (PREDICATE_1_NAME [?A1 ?A2 ... ?AN])
               (PREDICATE_2_NAME [?A1 ?A2 ... ?AN])
               ...)

  (:action ACTION_1_NAME
   [parameters (?P1 ?P2 ... ?PN)]
   [precondition PRECOND_FORMULA]
   [effect EFFECT_FORMULA]
  )

  (:action ACTION_2_NAME
   ...)

  ...)
```

Names (domain, predicate, action, etc.) may usually contain alphanumeric characters, hyphens ("-") and underscores ("_"), though there may be some planners that allow less.

Many of the reserved keywords start with a colon, so that newly invented keywords will not interfere

with names you have already used for other purposes in your domain and problem specifications.

Variables, such as parameters of predicates and actions, are distinguished by beginning with a question mark ("?").

Comments in a PDDL file start with a semicolon (";") and last to the end of the line.

Requirements

Because PDDL is a very general language and most planners support only a subset, domains may declare requirements (see the grammar below). The most commonly used requirements are:

`:strips`

The most basic subset of PDDL, consisting of STRIPS only.

`:equality`

The domain uses the predicate `=`, interpreted as equality.

`:typing`

The domain uses object types (see below).

`:adl`

The domain uses some or all of ADL (*i.e.* disjunctions and quantifiers in preconditions and goals, and quantified and conditional effects).

Type Definitions

In most current planners, objects can be declared to have specific types, such as "container", "location", "vehicle", and so on. If object types are to be used in a domain, the domain should first of all declare the requirement `:typing`.

All type names have to be declared before they are used (which usually means before the `:predicates` declaration). This is done with the following declaration:

```
(:types NAME1 ... NAME_N)
```

An example for the Dock Worker Robots domain:

```
(define (domain dock-worker-robots)
  (:requirements
    :strips
    :typing )

  (:types
    location ; there are several connected locations in the harbor
    pile      ; attached to a location, holds a pallet + a stack of containers
    robot     ; holds at most 1 container, only 1 robot per location
    crane     ; belongs to a location to pickup containers
    container)
)
```

We will return to object types when discussing predicate definitions, action definitions, and object specifications.

Predicate Definitions

Under `:predicates`, you define all predicates ("boolean state variables") to be used in the domain. The parameters variables used each predicate declaration have no other function than to specify the number of arguments that the predicate should have, *i.e.* the parameter names do not matter (as long as they are distinct). Predicates can have zero parameters (but in this case, the predicate name still has to be written within parentheses).

If you want to use typed parameters, each parameter is written as `?X - TYPE_OF_X`. A list of parameters of the same type can be abbreviated to `?X ?Y ?Z - TYPE_OF_XYZ`. Note that the hyphen between

parameter and type name has to be surrounded by whitespace. An example for the Dock Worker Robots domain:

```
(define (domain dock-worker-robots)
  (:requirements ...)
  (:types ...)
  (:predicates
    (adjacent ?l1 ?l2 - location)          ; can move from ?l1 directly to ?l2
    (attached ?p - pile ?l - location)      ; pile ?p attached to location ?l
    (belong ?k - crane ?l - location)       ; crane ?k belongs to location ?l

    (at ?r - robot ?l - location)          ; robot ?r is at location ?l
    (occupied ?l - location)               ; there is a robot at location ?l
    (loaded ?r - robot ?c - container)     ; robot ?r is loaded with container ?c
    (unloaded ?r - robot)                  ; robot ?r has no cargo

    (holding ?k - crane ?c - container)    ; crane ?k is holding container ?c
    (empty ?k - crane)                     ; crane ?k is not holding anything

    (in ?c - container ?p - pile)          ; container ?c is somewhere in pile ?p
    (top ?c - container ?p - pile)         ; container ?c is on top of pile ?p
    (on ?k1 ?k2 - container)               ; container ?k1 is on container ?k2
  )
)
```

Action Definitions

As shown above, an action is generally specified as follows:

```
(:action ACTION_1_NAME
  [:parameters (?P1 ?P2 ... ?PN)]
  [:precondition PRECOND_FORMULA]
  [:effect EFFECT_FORMULA]
)
```

All parts of an action definition except the name are, according to the spec, optional (although, of course, an action without effects is pretty useless). However, for an action that has no preconditions some planners may require an "empty" precondition, on the form `:precondition ()` (some planners may also require an empty `:parameter` list for actions without parameters). Example:

```
(define (domain dock-worker-robots)
  ...
  (:action move
    :parameters (?r - robot ?from ?to - location)

    :precondition (and (adjacent ?from ?to)
                      (at ?r ?from)
                      (not (occupied ?to)))

    :effect (and (at ?r ?to) (not (occupied ?from))
                (occupied ?to) (not (at ?r ?from)))
  )
)
```

Parameter types are declared in the same way as in predicate specifications. This ensures that planners do not try to apply actions using unexpected object types. For example, in Dock Worker Robots, we do not want a crane to be able to lift a robot and place it on top of a pile of containers. Cranes can only lift containers.

Note: Some planners only try to apply actions where all arguments are different, *i.e.* the same object may not be used as a value of two distinct parameters of the same action. Such planners might use actions such as `move(A,B)` and `move(B,A)`, but would not consider `move(A,A)` or `move(B,B)` to be a valid action. In this particular example, forbidding a move from one position to the same position

seems quite reasonable, but for other actions it may cause some difficulties. Consider the action `moveto(?piece,?destcolumn,?destrow)` for example. If we represent both columns and rows are represented by the constants `A/B/C/D/E/F/G/H`, a planner of the type discussed here would never use an action such as `moveto(bishop1,A,A)` or `moveto(pawn2,C,C)`, making it impossible to move to a position on the diagonal and making some problems unsolvable. See the **SlideTile domain definition** and the two problem definitions `eight01.pddl` and `eight01x.pddl` for an example of this problem and how to fix it.

Precondition Formulas

In a STRIPS domain, a precondition formula may be one of the following (see also the example above):

- ▶ An atomic formula:
`(PREDICATE_NAME ARG1 ... ARG_N)`
The predicate arguments must be parameters of the action (or constants declared in the domain, if the domain has constants).
- ▶ A conjunction of atomic formulas:
`(and ATOM1 ... ATOM_N)`

If the domain uses `:adl` or `:negated-precondition`, an atomic formula may also be of the form `(not (PREDICATE_NAME ARG1 ... ARG_N))`, specifying that the given fact must be false.

If the domain uses `:equality`, an atomic formula may also be of the form `(= ARG1 ARG2)`. Many planners that support equality also allow negated equality, which is written `(not (= ARG1 ARG2))`, even if they do not allow negation in any other part of the definition.

In an ADL domain, specified with `:adl`, a precondition may in addition be:

- ▶ A general negation, conjunction, disjunction, or implication:
`(not CONDITION_FORMULA)`
`(and CONDITION_FORMULA1 ... CONDITION_FORMULA_N)`
`(or CONDITION_FORMULA1 ... CONDITION_FORMULA_N)`
`(imply CONDITION_FORMULA1 CONDITION_FORMULA_2)`
- ▶ A quantified formula:
`(forall (?V1 ?V2 ...) CONDITION_FORMULA)`
`(exists (?V1 ?V2 ...) CONDITION_FORMULA)`

Effect Formulas

In PDDL, the effects of an action are not explicitly divided into "adds" and "deletes", as in some other planning languages. Instead, negative effects (deletes) are specified using a negation operator.

In a STRIPS domain, an effect formula may consist of the following (see also the example above):

- ▶ An added atom:
`(PREDICATE_NAME ARG1 ... ARG_N)`
The predicate arguments must be parameters of the action (or constants declared in the domain, if the domain has constants).
- ▶ A deleted atom:
`(not (PREDICATE_NAME ARG1 ... ARG_N))`
- ▶ A conjunction of effects:
`(and ATOM1 ... ATOM_N)`

The equality predicate (`=`) can of course not occur in an effect formula; no action can make two identical things be not identical, or vice versa!

In an ADL domain, an effect formula may in addition contain:

► A conditional effect:

```
(when CONDITION_FORMULA EFFECT_FORMULA)
```

The interpretation is that the specified effect takes place only if the specified condition formula is true in the state where the action is executed. Conditional effects are often but not always placed within quantifiers.

► A universally quantified formula:

```
(forall (?V1 ?V2 ...) EFFECT_FORMULA)
```

The PDDL Problem Definition

The problem definition first specifies its name and which domain it belongs to. It then specifies the objects present in the problem instance, the initial state of the world, and the goal.

The format of a (simple) problem definition is:

```
(define (problem PROBLEM_NAME)
  (:domain DOMAIN_NAME)
  (:objects OBJ1 OBJ2 ... OBJ_N)
  (:init ATOM1 ATOM2 ... ATOM_N)
  (:goal CONDITION_FORMULA)
)
```

The object list can be typed or untyped. A typed example from Dock Worker Robots:

```
(define (problem dwr-problem-1)
  (:domain dock-worker-robots)
  (:objects
    r1 - robot
    loc1 loc2 - location
    k1 - crane
    p1 p2 - pile
    c1 c2 c3 pallet - container)
  ...
)
```

The initial state description (the `:init` section) is simply a list of all the ground (variable-free) atoms that are true in the initial state. All other atoms are by definition false. For example:

```
(define (problem dwr-problem-1)
  (:domain dock-worker-robots)
  (:objects ...)
  (:init
    (attached p1 loc1) (in c1 p1) (on c1 pallet) (in c3 p1) (on c3 c1) (top c3 p1)
    (attached p2 loc1) (in c2 p2) (on c2 pallet) (top c2 p2)
    (belong crane1 loc1) (empty crane1)
    (at r1 loc2) (unloaded r1) (occupied loc2)
    (adjacent loc1 loc2) (adjacent loc2 loc1)
  )
)
```

The goal description is not a state or list of literals, but is instead expressed as a formula of the same form as an action precondition. Example:

```
(define (problem dwr-problem-1)
  (:domain dock-worker-robots)
  (:objects ...)
  (:init ...)
  (:goal (and (in c1 p2) (in c3 p2))))
```

Typing

PDDL has a (very) special syntax for declaring parameter and object types. If types are to be used in a domain, the domain should first of all declare the requirement `:typing`.

Second, the type names have to be declared before they are used (which usually means before the `:predicates` declaration). This is done with the declaration

```
(:types NAME1 ... NAME_N)
```

Then, to declare the type of a parameter of a predicate or action one writes `?X - TYPE_OF_X`. A list of parameters of the same type can be abbreviated to `?X ?Y ?Z - TYPE_OF_XYZ`. Note that the hyphen between parameter and type name has to be "free-standing", *i.e.* surrounded by whitespace.

The syntax is the same for declaring types of objects in the problem definition.

Common problems

- ▶ If there are errors in your domain or problem definitions, some planners may silently accept them anyway, but give unpredictable results. If you are having problems finding plans, please try running Fast Downward on the definitions – it may point out plain errors that another planner has missed.

```
/courses/TDDC17/sw/fdlog/fast-downward.py DOMAIN PROBLEM CONFIGURATION
```

- ▶ Be careful about mixing *type predicates* and *types*. Type predicates and types are two different solutions that should not be used together, and there is no information flow from one to the other. This could lead to situations where you can't find plans because you have declared an object to be of *type* `block` without declaring that it satisfies the *type predicate* `(block ?b)`, or vice versa.

As we discussed above, many planners support actual types, which work very much like types in programming languages: Parameter variables are declared to be of particular types, objects are defined as belonging to particular types, and so on. If you use actual types, and declare a list of objects as (for example) `B1 B2 B3 - block`, then this will *not* result in a type predicate that you can use as `(block ?b)`.

And if you do use type predicates, by declaring a predicate called `block`, then this predicate will have to be initialized in the ordinary initial state, by a declaration such as `(block B1) (block B2) (block B3)`.